LogiSync — AWS Architecture

Real-time delivery risk detection and proactive intervention

Proposed production architecture | Independent product case study

1. Architecture Diagram

The diagram presents a proposed AWS architecture for ingesting delivery events, detecting potential risks, enabling human-reviewed interventions, and measuring outcomes.



2. Architecture Overview

I recommend a serverless, event-driven architecture rather than starting with a large microservices platform. It supports the initial product hypothesis while keeping the first production iteration focused on reliable data ingestion, explainable risk detection, and human-approved actions.

Layer	Proposed AWS services	Responsibility
Web experience	Amazon S3, CloudFront, AWS WAF	Serve and protect the operations dashboard.
Identity and access	Amazon Cognito, AWS IAM	Authenticate users and enforce least-privilege access.
APIs and business logic	Amazon API Gateway, AWS Lambda	Receive events, validate requests and execute product logic.

Event processing	Amazon EventBridge, Amazon SQS, AWS Step Functions	Process events asynchronously, retry failures and orchestrate workflows.
Operational data	Amazon DynamoDB, Amazon S3	Store current delivery/exception state and larger evidence artifacts.
Risk detection	Lambda-based rules engine	Flag potentially at-risk deliveries using transparent, testable rules.
Optional AI assistance	Amazon Bedrock; future ML evaluation	Support grounded summaries; evaluate ML only if it adds value over rules.
Notifications	Amazon SNS and/or Amazon SES	Deliver authorized alerts and customer communications through verified integrations.
Observability and security	Amazon CloudWatch, AWS CloudTrail, AWS KMS, AWS Secrets Manager	Monitor behavior, audit activity, encrypt data and protect credentials.

3. Proposed Event Flow

1. Delivery systems, telematics and field reports provide delivery status, timestamps and exception signals.
2. Amazon API Gateway and AWS Lambda validate incoming events and update operational records.
3. Amazon EventBridge routes relevant events; Amazon SQS buffers processing and supports retries, with a dead-letter queue for persistent failures.
4. AWS Step Functions coordinates multi-step workflows where needed.
5. A transparent rules engine identifies potential risks and stores the evidence and alert reason.
6. The operations dashboard presents alerts with source, timestamp, freshness and uncertainty.
7. An authorized coordinator reviews the alert and approves any consequential intervention.
8. The system records the action and delivery outcome for measurement and future evaluation.

4. Key Product and Architecture Decisions

Rules before ML: Start with explainable rules using available delivery-status signals. Compare any future ML model against this baseline and introduce it only if incremental value is demonstrated.

Human approval: A risk alert does not automatically reschedule a delivery or send an unverified customer commitment. Consequential actions require an authorized human decision.

Separate operational state from evidence: Use DynamoDB for operational state and S3 for suitable evidence artifacts, with retention and access rules defined before production.

Event-driven processing: Use asynchronous processing for event ingestion and workflow steps so transient downstream failures can be retried and isolated.

Grounded AI assistance: Amazon Bedrock is optional and should assist with summaries of approved evidence rather than independently decide operational outcomes.

5. Recommended MVP Scope

- Secure operations dashboard and defined user roles.
- Delivery-event ingestion through a controlled API or data import.
- Rules-based risk detection with visible reasons and supporting evidence.

- Exception queue with human review and recorded decisions.
- Human-approved intervention workflow and audit records.
- Basic monitoring, retries, error handling and shadow-mode evaluation.

6. Production Readiness Considerations

Data contracts: Define event schemas, identifiers, timestamps, duplicate handling, freshness and original delivery-commitment retention.

Security and privacy: Apply least-privilege IAM, appropriate authentication, encryption, data retention controls and auditability.

Reliability: Define retries, idempotency, dead-letter handling, alarms, recovery procedures and rollback.

AI governance: Maintain evaluation datasets, model/version records, grounding controls, human review and drift monitoring where applicable.

Cost: Estimate event volume, compute, storage, logs, notifications and optional AI inference before deployment.

Integrations: Validate supported TMS/OMS interfaces, credentials, external-system failure behavior and authorization.

7. Prototype vs. Production Boundary

LogiSync is currently an interactive prototype with some working flows; logistics data and integrations are mocked or simulated. This document describes a proposed production architecture. It does not claim that the AWS services, live integrations, security controls, reliability controls, ML capabilities or business outcomes have already been implemented or verified.

8. Summary

I designed LogiSync around a proposed serverless, event-driven AWS architecture for delivery-event ingestion, risk detection, exception review and human-approved interventions.

The proposed stack uses API Gateway and Lambda for APIs and business logic; EventBridge, SQS and Step Functions for asynchronous processing; DynamoDB for operational state; and S3 for evidence and reporting artifacts. Cognito supports authentication, while CloudWatch, CloudTrail, KMS and Secrets Manager address observability and security. The initial risk-detection approach is rules-based and explainable.

Bedrock is an optional extension for grounded exception summaries, while historical-data ML remains subject to data readiness and comparative evaluation. Live integrations, production controls and business impact remain subject to implementation and validation.